

输入/输出

一.输入输出概述

1. I/O 是程序的一个重要的组成部分。程序运行所需要的数据往往要从外设中得到。程序的运行结果往往也要输出到外设中。
2. 在 $C++$ 中, I/O 操作不是语言定义的成分, 而是由具体的实现作为标准库的功能来提供。
3. 在 $C++$ 中, 输入输出是一种基于字节流的操作:
 - 在进行输入操作时, 可把输入的数据看成逐个字节地从外设流入到计算机内存。
 - 在进行输出操作时, 则把输出的数据看成逐个字节地从内存中流出外设。
4. 在 $C++$ 的标准库中, 提供了两大类 I/O 操作:
 - 基于字节流的 I/O 操作
 - 基于 $C++$ 基本数据类型数据的 I/O 操作
5. 输入/输出操作往往是带内存缓冲区的:
 - 输出时, 输出的数据是先放内存缓冲区中, 缓冲区满了之后再实际输出到外设。
 - 输入时, 先从外设输入一大块内容放入缓冲区, 然后从缓冲区中逐步输入数据。
6. $C++$ 输入输出的实现途径
 - 过程式--通过从C语言保留的函数库中的输入输出函数来实现: `printf`、`scanf`
 - 面向对象--通过 $C++$ 的 I/O 类库中的类/对象以及重载的操作符“<<”和“>>”来实现。如:
 - `ostream`、`cout`、<<
 - `istream`、`cin`、>>
7. `printf`、`scanf` 的缺陷:

参数个数和类型不固定, 编译时刻不能进行严格的参数类型检查 (不是强类型), 使用不当会导致运行时刻的错误。

需要用格式串来指出后面要输出的数据的类型。如:

```
1 int i;  
2 double j;  
3 scanf("i=%d,j=%lf",&i,&j);  
4 printf("i=%d,j=%lf\n",i,j);
```

编译时刻一般不对格式串中指定的数据类型和个数与所提供的数据的实际类型和个数之间是否一致进行的检查, 当格式串描述与实际数据不一致时会导致运行时刻的错误。

8. `cout`、`cin` 的优势:

不需要单独指定数据的类型和个数, 编译时刻根据数据本身来决定操作的类型和个数, 这样可避免与类型和个数相关的错误。
9. I/O 的分类:
 - 面向控制台的
 - 面向文件的
 - 面向字符串变量的
10. $C++$ 的 I/O 类库中基本的类:

其中面向字符串变量的 *I/O* 已经被 *STL* 中的 `string` 替代。



在进行输入/输出时，首先创建某个 *I/O* 类的对象，然后：

- 可调用该对象类的成员函数进行基于字节流的输入/输出操作。如：

```
1 //输入操作
2 istream in();
3 in.get(ch); //读入一个字节，ch为一个字符变量
4 in.read(p,100); //读入100个字节，p是内存空间首地址
5 //或：
6 in.get(ch).read(p,100);
7
8 //输出操作
9 ostream out();
10 out.put(ch); //输出一个字节，ch为一个字符变量
11 out.write(p,100); //把p指向的内存中100个字节输出
12 //或：
13 out.put(ch).write(p,100);
```

- `istream`类和`ostream`类以及它们的派生类还分别重载了操作符“>>”(抽取)和“<<”(插入)，用它们可以进行 *C++* 基本类型数据的输入/输出操作。如：

```
1 //输入操作
2 istream in(...);
3     in >> x; //x是一个基本数据类型的变量
4     in >> y; //y是一个基本数据类型的变量
5 //或
6     in >> x >> y;
7
8 //输出操作
9     ostream out(...);
10     out << e1; //e1是一个表达式
11     out << e2; //e2是一个表达式
12 //或
13     out << e1 << e2;
```

二.面向控制台的 I/O

1. 在I/O类库中预定义了四个I/O对象，可以直接用来进行控制台的输入/输出操作:

- cin (istream类的对象)：对应着计算机系统的标准输入设备。（通常为键盘）
- cout (ostream类的对象)：对应着计算机系统的标准输出设备。（通常为显示器）
- cerr和clog (ostream类的对象)：对应着计算机系统用于输出特殊信息（如程序错误信息）的设备。（通常也对应着显示器，但不受输出重定向的影响）。cerr为不带缓冲的，clog为带缓冲的。

在进行控制台输入输出时，程序中需要有下列的包含命令：

- #include

2. 控制台输出操作：

- 用插入操作符 (<<) 进行基本数据类型数据的输入。如：

```
1  #include <iostream>
2  using namespace std;
3  .....
4  int x;
5  float f;
6  char ch;
7  int *p=&x;
8  .....
9  cout << x ; //输出x的值。
10 cout << f; //输出f的值。
11 cout << ch; //输出ch的值。
12 cout << "hello"; //输出字符串"hello"。
13 cout << p; //输出变量p的值，即，变量x的地址。
14 //或
15 cout << x << f << ch << "hello" << p;
```

输出指向字符的指针时，并不是输出指针的值，而是输出它指向的字符串。例如：

```
char *p;
```

```
.....
```

```
cout << p; //输出p指向的字符串
```

如果要输出字符指针的值，需要把它转换成其它类型的指针。例如：

```
cout << (void *)p; //输出p的值
```

3. 输出格式控制：

为了对输出格式进行控制，可以通过输出一些操纵符 (manipulator) 来实现。如：

```
1  #include <iostream>
2  #include <iomanip> //操纵符声明的头文件
3  using namespace std;
4  .....
5  int x=10;
6  cout << hex //以十六进制输出后面的值
7      << x
8      << endl; //输出换行符
```

操纵符	含义
endl	输出换行符，并执行flush操作
flush	使输出缓存中的内容立即输出
dec	十进制输出
oct	八进制输出
hex	十六进制输出
setprecision(int n)	设置浮点数的精度（由输出格式决定是有效数字的个数还是小数点后数字的位数）
setiosflags(long flags)/resetiosflags(long flags)	设置/重置输出格式，flags的取值可以是： ios::scientific(以指数形式显示浮点数)，ios::fixed(以小数形式显示浮点数)等

对于浮点数（float、double和long double）：

- 当输出格式为ios::scientific或ios::fixed时，操纵符setprecision用于设置浮点数小数点后面的位数；
- 当输出格式为自动方式（既不是ios::scientific也不是ios::fixed，或两者都设了）时，操纵符setprecision用于设置浮点数有效数字的个数，这时的输出格式根据有效数字自动确定。

除了用插入操作符（<<）进行基本数据类型数据的输出外，还可以用ostream类的成员函数进行基于字节的输出操作。例如：

```

1 //输出一个字节。
2 ostream& ostream::put(char ch);
3 cout.put('A');
4 //输出p所指向的内存空间中count个字节。
5 ostream& ostream::write(const char *p,int count);
6 char info[100];
7 int n;
8 .....
9 cout.write(info,n);

```

4. 控制台输入操作：

用抽取操作符（>>）进行基本数据类型数据的输入。

```

1 #include <iostream>
2 using namespace std;
3 .....
4 int x;
5 double y;
6 char str[10];
7 cin >> x; cin >> y; cin >> str;
8 //或者
9 cin >> x >> y >> str;

```

在输入时，各个数据之间一般要用空白符（空格、\t、\n）分开：

- 输入一个数据前，先跳过空白符

◦ 输入一个数据的过程中，碰到空白符或当前数据类型不允许的字符时结束当前数据的输入
可以通过一些操纵符来控制输入的行为，例如：

```
1 char str[10];
2 cin >> setw(10) >> str; //把输入的字符串和一个'\0'
3                          //放入str中，最多输入9个字符
```

除了用抽取操作符 (>>) 进行基本数据类型数据的输入外，还可以用istream类的成员函数进行基于字节的输入操作。例如：

```
1 //输入一个字节。
2 istream& istream::get(char &ch);
3 //输入count个字节至p所指向的内存空间中。
4 istream& istream::read(char *p,int count);
5 //输入一个字符串。输入过程直到输入了count-1个字符或
6 //遇到delim指定的字符为止，并自动加上一个'\0'字符。
7 istream& istream::getline(char *p, int count,
8                           char delim='\n');
```

用上述函数可以输入空白符！例如：

```
1 char str[10];
2 cin.getline(str,10); //可以输入带空格的字符串
```

5. 操作符">>"和"<<"的重载

标准库中重载的操作符">>"和"<<"只能对基本数据类型的数据进行输入/输出。可以针对自定义的类进一步重载操作符">>"和"<<"，从而实现对某个类的对象进行输入/输出操作。

```
1 //针对类A重载操作符"<<"，可实现A类对象的输出：
2 class A
3 {     int x,y;
4     public:
5         .....
6     friend ostream& operator << (ostream& out, const A &a);
7 };
8 ostream& operator << (ostream& out, const A &a)
9 {     out << a.x << ',' << a.y;
10     return out;
11 }
12 .....
13 A a1,a2;
14 cout << a1 << endl << a2 << endl;
15
```

三.面向文件的 *I/O*

1. 需求：程序运行结果有时需要永久性地保存起来，以供其他程序或本程序下一次运行时使用。程序运行所需要的数据也常常要从其他程序或本程序上一次运行所保存的数据中获得。
2. 用于永久性保存数据的设备称为外部存储器（外存），如：磁带、磁盘、光盘等。在外存中保存数据的方式通常有两种：文件、数据库。
3. 文件数据的存储方式：

- 文本方式 (text) :
 - 只包含**可显示的字符**和有限的几个**控制字符** (如: \r、\n、\t等) 的**编码**。
 - 一般用于存储具有“行”结构的**文字**数据, 数据可用记事本等软件打开。
- 二进制方式 (binary) :
 - 包含任意的**没有显式含义的纯二进制字节**。
 - 一般用于存储任意结构的数据, 数据由使用它的应用程序来解释。

如: 对于一个整数1234567

文本方式存储: 按字符串依次把字符1、2、3、4、5、6、7的ASCII码写入文件 (7个字节) 。

二进制方式存储: 按整型数1234567的int型机内表示 (0x00 12D6 87)把它逐个字节依次写入文件 (4个字节) 。

4. 文件的读写过程:

- 在外存中, 每个文件都有一个名字 (文件名) (操作系统一般采用树形的目录结构来管理外存中的文件) 。
- 对文件数据进行读写的过程:
 - 打开文件, 把程序内部的一个表示文件的变量/对象与外部的一个实际文件关联起来, 并创建内存缓冲区。
 - 文件读/写, 存取文件中的内容。
 - 关闭文件, 把暂存在内存缓冲区的内容写入到文件中, 并归还打开文件时申请的内存资源 (包括内存缓冲区) 。

5. 文件的位置指针:

- 在C++中, 把文件看成一系列字节所构成的字节串, 对文件中数据的操作 (输入/输出) 通常是逐个字节顺序进行, 因此称为流式文件。
- 每个打开的文件都有一个内部 (隐藏) 的位置指针, 它指出文件的当前读写位置。
- 进行读/写操作时, 没每读入/写出一个字节, 文件位置指针会自动往后移动一个字节位置。

6. 在利用I/O类库中的类进行文件的输入输出时, 程序中需要包含下面的头文件:

```
#include
```

```
#include
```

7. 文件的输出操作:

- 打开文件: 创建一个ofstream类的对象, 并建立与外部某个文件之间的联系。
 - 直接方式:

```
1 ofstream out_file(<文件名> [, <打开方式>]);
2 //例如:
3 ofstream out_file("d:\\myfile.txt", ios::out);
4
```

- 间接方式:

```
1 ofstream out_file;
2 out_file.open(<文件名> [, <打开方式>]);
3 //例如:
4 out_file.open("d:\\myfile.txt", ios::out);
5
```

■ 打开方式

- `ios::out`: 打开一个外部文件用于写操作。（文件指针位置位于文件的头）。如果外部文件已存在，则首先把它的内容清除；否则，先创建该外部文件。`ios::out`是默认打开方式。
- `ios::app`: 打开一个外部文件用于添加操作。（不清除文件内容，文件位置指针位于文件末尾）。如果外部文件不存在，则先创建该外部文件。

■ `ios::out|ios::binary`或`ios::app|ios::binary`

- 按二进制方式打开文件。（默认的是文本方式）
- 两种方式区别：对以文本方式打开的文件，当输出的字符为
时，在某些平台上将自动把它转换成
和
两个字符写入外部文件。对以二进制打开的文件，对输出的字节不做任何转换，原样输出。

■ 判断打开操作是否成功：

```
1  if (!out_file.is_open()) //或:
    out_file.fail()
2
3                      //或: !out_file
4  { ..... //失败处理
5  }
```

- 输出数据：文件成功打开后，可以使用插入操符“<<”或`ofstream`类的一些成员函数来进行文件数据的输出操作，例如：

```
1  int x=12;
2  double y=12.3;
3  .....
4  //以文本方式输出数据
5  ofstream out_file("d:\\myfile.txt",ios::out);
6  if (!out_file) exit(-1);
7  out_file << x << ' ' << y << endl; //输出: 12 12.3
8
9  //或
10 //以二进制方式输出数据
11 ofstream out_file("d:\\myfile.dat",ios::out|ios::binary);
12 if (!out_file) exit(-1);
13 out_file.write((char *)&x,sizeof(x)); //输出: 4个字节
14 out_file.write((char *)&y,sizeof(y)); //输出: 8个字节
15
```

- 文件输出操作结束时，要使用`ofstream`的成员函数`close`关闭文件：`out_file.close()`;

目的：把文件内存缓冲区的内容写到磁盘文件中

8. 文件的输入操作：

- 打开文件：创建一个`ifstream`类的对象，并与外部文件建立联系。

- 直接方式：`ifstream in_file(<文件名> [<打开方式>]);`

- 间接方式：`ifstream in_file;`

`in_file.open(<文件名> [<打开方式>]);`

- 打开方式：

- `ios::in`：打开一个外部文件用于读操作。（默认）

- `ios::in | ios::binary`：按二进制方式打开文件。（默认为文本方式）对以文本方式打开的文件，当读入的字符为连续的'\r'和'\n'两个字符时，在某些平台上（如：DOS和Windows平台）将自动转换成一个字符'\n'。以二进制方式打开的文件，对输入的字节不做任何转换，原样输入。
- 打开文件时要判断打开是否成功。判断方式与文件输出打开操作的判断一样。
- 文件成功打开后，可以使用抽取操作符">>"或`ifstream`类的一些成员函数来进行文件的输入操作，例如：

```

1  int x;
2  double y;
3  .....
4  //以文本方式输入数据
5  ifstream in_file("D:\\myfile.txt",ios::in);
6  if (!in_file) exit(-1);
7  in_file >> x >> y;
8
9  //以二进制方式输入数据
10 ifstream in_file("D:\\myfile.dat",ios::in|ios::binary);
11 if (!in_file) exit(-1);
12 in_file.read((char *)&x,sizeof(x));
13 in_file.read((char *)&y,sizeof(y));
14

```

- 读取数据过程中有时需要判断是否正确读入了数据（尤其是在文件末尾处）。判断是否正确读入了数据，可以调用`ios`类的成员函数`fail`来实现：`bool ios::fail() const`；该函数返回`true`表示文件操作失败；返回`false`表示操作成功。
- 文件输入操作结束后，要使用`ifstream`的一个成员函数`close`关闭文件：`in_file.close()`；
- 注意：以文本方式输出的文件要以文本方式输入；以二进制方式输出的文件要以二进制方式输入！以文本方式读写的文件要以文本方式打开；以二进制方式读写的文件要以二进制方式打开！

9. 以二进制方式存取文件不利于程序的兼容性和可移植性。例如：

- 在不同计算机平台上，整型数的各字节在内存中的存储次序可能不一样（先存高位字节或低位字节），因此，写到文件中的次序也不一样。
- 在不同的编译环境下，同样的结构类型数据的尺寸（字节数）可能不一样（根据对成员地址对齐的不同要求），因此，写到文件中的字节数也不一样。

10. 如果需要打开一个既能读入数据、也能输出数据的文件，则需要创建一个`fstream`类的对象。

- 文件内部有两个位置指针，一个用于读，另一个用于写。
- 在创建`fstream`类的对象并建立与外部文件的联系时，文件打开方式应为下面之一：

`ios::in | ios::out`（可在文件任意位置写）

`ios::in | ios::app`（只能在文件末尾写）

11. 文件的随机存取：

- 对于流式文件，要读入/写出第`n`个字节，一般需要先读入/写出前`n-1`个字节。
- 为了能够随机读写文件中的数据，可以显示地指出读写的位置。下面的操作用来指定文件内部读指针的位置：

```
1 istream& istream::seekg(<位置>); //指定绝对位置
2 istream& istream::seekg(<偏移量>,<参照位置>); //指定相对位置
3 streampos istream::tellg(); //获得读操作的指针位置
4
```

下面的操作来指定文件内部写指针的位置:

```
1 ostream& ostream::seekp(<位置>); //指定绝对位置
2 ostream& ostream::seekp(<偏移量>,<参照位置>); //指定相对位置
3 streampos ostream::tellp(); //获得写操作的指针位置
4
```

- <参照位置>可以是: ios::beg (文件头) , ios::cur (当前位置) 和ios::end (文件尾) 。
- 文件的随机存取一般用于以二进制方式存贮的文件。